FACULTY OF ECONOMICS AND BUSINESS ADMINISTRATION

InClass Kaggle Competition: Price Suggestion Prediction for Online Resale

Technical Report

Course: Machine Learning

Teacher: Prof. Dr. Dries Benoit
Lukas De Kerpel (TA)

Students: Team 11

Febe De Smet	02103805	Febe.desmet@ugent.be
Henri Bolliou	02112399	Henri.bolliou@ugent.be
Liam Kielemoes	02111046	Liam.kielemoes@ugent.be
Lucas Minnaert	02113094	Lucas.minnaert@ugent.be

Academic year: 2025–2026

GHENT UNIVERSITY

Contents

1 Introduction **3**

2 Data Preparation **3**

2.1 Data splitting . . . 3

2.2 Data cleaning . . . 4

2.2.1 String manipulation . . . 4

2.2.2 Missing values . . . 4

2.2.3 Outliers . . . 4

2.3 Feature engineering . . . 5

2.3.1 Structured features . . . 5

2.3.2 Text features . . . 5

2.3.3 Final feature matrices . . . 6

3 Model Training **7**

3.1 Linear baselines . . . 7

3.1.1 Linear regression . . . 7

3.1.2 Ridge regression . . . 7

3.1.3 Lasso regression . . . 8

3.2 Generalized additive models . . . 8

3.2.1 Smoothing splines . . . 8

3.3 Tree-based models . . . 9

3.3.1 Decision tree . . . 9

3.3.2 Bagging . . . 10

3.3.3 Random forest . . . 10

3.3.4 BART . . . 11

3.3.5 Gradient boosting . . . 11

3.3.6 XGBoost . . . 12

3.3.7 LightGBM . . . 13

3.4 Support vector regression . . . 14

3.4.1 Linear kernel . . . 14

3.4.2 Non-linear kernel . . . 14

3.5 Neural networks . . . 15

3.5.1 Multi-layer perceptron . . . 15

4 Model Evaluation **17**

5 Conclusion **17**

A Preprocessing and Feature Engineering **19**

B Models **20**

B.0.1 GAM 20

C Evaluation **21**

1 Introduction

This project predicts second-hand product prices on an online marketplace from structured attributes and free-text fields. We provide a reproducible pipeline that cleans the raw data, encodes all variables, and exports model-ready feature matrices for downstream training and evaluation.

2 Data Preparation

The original training set (`train.csv`) contains 489,985 observations with product metadata and free-text fields, while the test set (`test_X.csv`) has the same structure except for the missing target variable `price`. Table 1 summarises the variables used in the modelling pipeline.

Preprocessing was organised into three stages (medaillon structure):

1. cleaning and standardising the raw training and test data (Bronze → Silver);
2. constructing a fixed internal validation split for model selection;
3. feature engineering and encoding of all variables (Silver → Gold).

Column name	Type	# unique values
index	numeric	489 985
name	text	434 151
item_condition_id	categorical/ordinal	5
brand_name	text/categorical	3 392
shipping	categorical	2
item_description	text	425 183
categorie_1	text/categorical	50
categorie_2	text/categorical	217
categorie_3	text/categorical	864

Table 1: Overview of the main variables in the training data set.

Note. These columns reflect the dataset after all cleaning steps.

2.1 Data splitting

We created an 80/20 train/validation split with a fixed seed before feature engineering to prevent leakage. All transformations were fit on the training subset and applied unchanged to validation/test. Matrices and targets were stored as .rds in the Gold layer (memory efficiency).

2.2 Data cleaning

The raw dataset contained substantial noise, inconsistent formatting and non-standard missing values across both structured and unstructured fields. To enable reliable feature engineering, all inputs were systematically standardised prior to modelling.

2.2.1 String manipulation

Text fields showed considerable variation (spelling, punctuation, casing). Standardisation was applied to ensure consistent vocabulary construction and brand extraction.

The `category_name` field encoded a three-level hierarchy using irregular separators. After a small set of normalisations (e.g. "t-shirt" → "tshirt"), entries were split on common delimiters or, when absent, tokenised via simple heuristics (e.g. PascalCase detection). Products were mapped to `categorie_1–3`, with missing levels stored as `NA` and later one-hot encoded.

2.2.2 Missing values

Non-standard missing indicators (empty strings, "–", "?", "N/A") were mapped to `NA`.

The `brand_name` column contained heterogeneous spellings, accented characters and many implicit missing values. After normalising text (lowercasing, accent removal and punctuation harmonisation), we constructed a lookup table of canonical brand forms. We imputed missing values by matching these terms first in the product title and then in the description. Remaining gaps were assigned to a dedicated "`unknown`" category.

For `shipping` only two valid values occurred, all other entries were set to `unknown`. Irregular values in `item_condition_id` were set to `NA` and later mapped to a dedicated category (0).

Seventeen rows with invalid or empty prices were removed. After encoding, all features were numeric. Any remaining `NA` values (mainly in sparse text features) were set to zero.

2.2.3 Outliers

Description lengths and prices were inspected for extreme values. A small number of listings had disproportionately long descriptions, these were truncated to 90 words to stabilise text vectorisation without reducing information.

Prices showed a heavy right tail. Instead of removing high-value listings, we applied a log-transform $\log(\texttt{price} + 1)$, reducing the leverage of extreme values while retaining the full distribution. The original and transformed price distributions are shown in Appendix A.

2.3 Feature engineering

The final feature matrices combine structured attributes and text-derived features. All transformations were fitted on the training subset and applied unchanged to the validation and test sets to avoid data leakage.

2.3.1 Structured features

From `shipping` we constructed a three-level factor (buyer, seller, unknown). Missing values in `item_condition_id` were mapped to a separate level 0.

The hierarchical category fields `categorie_1–3` were used as categorical predictors. To limit sparsity, rare levels in `categorie_2` (<4 listings) and `categorie_3` (<50 listings) were collapsed into `other_cat2` and `other_cat3`. Remaining missing values were mapped to an explicit `Unknown_cat` level and all categories were one-hot encoded.

Because the cleaned dataset still contained many infrequent brands, `brand_name` levels occurring fewer than 50 times were grouped into a single `other_brand` category.

We added three simple numeric features: description word count, a missing description indicator, and title length. The continuous variables were standardised (z-scores based on the training subset).

2.3.2 Text features

Two text fields were constructed: a combined text field (`text_all`) containing name, truncated description and category name, and a cleaned version of the title (`name_clean`) for character-based features.

Word-level features were built with `text2vec`: tokenisation with Porter stemming, a pruned unigram/bigram vocabulary (min. 20 documents, max. 50% document frequency, capped at 8,000 terms), and a TF–IDF transformation. The fitted model was reused for the validation and test sets.

To capture subword patterns and mitigate spelling noise, we added character 3-gram features derived from `name_clean`. Only 3-grams appearing in at least 30 titles (max. 2,000) were retained, and each listing was represented by a sparse binary vector.

The TF–IDF matrix and character 3-gram matrix were concatenated and reduced with a rank-100 truncated SVD using `irlba`. The resulting 100 latent components (`svd_text_1–svd_text_100`) captured most of the variance and were used as dense text features for all models. A schematic overview is given in Figure 1; cumulative explained variance is shown in Appendix A.

2.3.3 Final feature matrices

All feature blocks were combined into a single sparse design matrix containing:

- 100 SVD-based text components;
- one-hot encoded categorical variables (shipping, item condition, categories, brand);
- three numeric features (description length, missing description, title length).

The resulting matrices ($X_{\text{train}}, X_{\text{val}}$) are highly sparse. The target used for modelling is the log-transformed price.

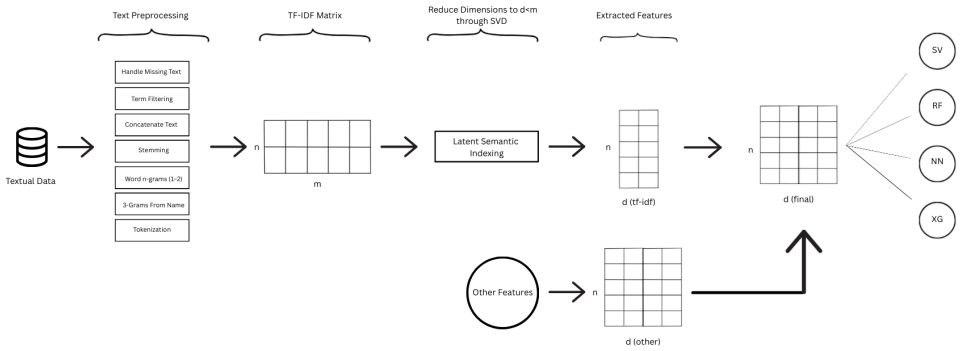


Figure 1: Feature engineering proces.

We also experimented with several handcrafted feature extensions, including target encodings (e.g. price-conditioned category and brand averages) and sentiment scores extracted from the text. Neither these engineered features nor alternative text embeddings (word2vec, GloVe) produced meaningful performance gains. Consequently, we retained the simpler and more effective TF–IDF + SVD representation as our final feature matrix.

3 Model Training

All models made use of the same validation strategy. Hyperparameters were tuned on the training set via k-fold cross-validation (with early stopping where applicable), after which each model was retrained on the full training set with the selected configuration and evaluated once on the validation set. The final submission model was retrained on train+validation and used to predict the test set.

3.1 Linear baselines

3.1.1 Linear regression

Specification

Ordinary least squares (OLS) was used as an unregularized linear baseline on the full feature matrix. Due to rank deficiency in the TF–IDF design matrix, a small number of non-identifiable coefficients were returned as NA and were set to zero when generating predictions.

Results

OLS achieved a validation RMSE of 32.91. Despite the large feature space, the model performs competitively, indicating that linear structure in the data is strong and that overfitting remains limited even without shrinkage.

3.1.2 Ridge regression

Specification

Ridge extends OLS with an ℓ_2 penalty that shrinks coefficients and stabilizes estimation in high-dimensional data. The model minimizes a penalized least squares objective with regularization parameter λ.

Hyperparameter tuning

Ridge was tuned over a 50-value logarithmic grid in $[10^{-4}, 10^{1}]$ using five-fold cross-validation. The selected value is shown in Table 2.

Name	**Range**	**Selected value**
lambda λ	$[10^{-4},\ 10^{1}]$ (50 values)	5.18×10^{-4}

Table 2: Ridge regression hyperparameters.

Results

Ridge reached a validation RMSE of 32.92, nearly identical to OLS. The small optimal penalty indicates limited benefit from variance reduction,

3.1.3 Lasso regression

Specification

Lasso introduces an ℓ_1 penalty that induces sparsity by shrinking many coefficients to zero. This can reduce variance but may remove useful signal in dense, high-dimensional problems.

Hyperparameter tuning

The same grid of 50 λ values was used. Five-fold cross-validation selected the value reported in Table 3.

Name	Range	Selected value
lambda λ	$[10^{-4}, 10^{1}]$ (50 values)	1.0×10^{-4}

Table 3: Lasso regression hyperparameters.

Results

Lasso achieved a validation RMSE of 32.94, slightly worse than both OLS and ridge. The additional shrinkage appears to remove predictive information,

3.2 Generalized additive models

3.2.1 Smoothing splines

Specification

A GAM was fitted using cubic smoothing splines for predictors showing clear non-linear patterns. Exploratory linear-versus-spline comparisons (Appendix B.0.1) indicated strong non-linearity for `svd_text_2`, `svd_text_6`, `svd_text_24`, `svd_text_5`, and `desc_word_count`, while the remaining predictors in the reduced feature set were modelled linearly.

Hyperparameter tuning

Spline flexibility (df_λ) was tuned with 3-fold cross-validation over $df_\lambda \in [2, 8]$ on the reduced predictor set. All smooth terms selected $df_\lambda = 8$ (Table 4).

Predictor	Range df_λ	Selected value df_λ
svd_text_2	[2, 8]	8
svd_text_6	[2, 8]	8
svd_text_24	[2, 8]	8
svd_text_5	[2, 8]	8
desc_word_count	[2, 8]	8

Table 4: Smoothing spline hyperparameters selected via cross-validated RMSE.

Results

The tuned GAM fitted on the full training set achieved a validation RMSE of 32.87, improving over its linear counterpart. All spline terms remained strongly significant (Appendix B.0.1).

3.3 Tree-based models

3.3.1 Decision tree

Specification

A single decision tree was implemented using the `rpart` package with ANOVA splitting. The tree was grown on the full training dataset.

Hyperparameter tuning

Tuning focused on structural control to manage complexity. The key parameter, Maximum Depth (maxdepth), was used as the structural constraint to indirectly tune the number of terminal nodes. The low Complexity Parameter (cp=0.00001) allowed the tree to grow freely to these imposed limits. This robust tuning process was executed on a 50,000-sample subset using external validation to determine the optimal maxdepth configuration, as detailed in Table 5.

Name	Range	Selected value
Terminal nodes	[10, 15, 20, 25]	25

Table 5: Decision tree hyperparameters.

Results

The selected configuration (Table 5) achieved a validation RMSE of 34.85.

3.3.2 Bagging

Specification

Bagging was implemented using the `randomForest` package, with m_{try} set equal to the total number of predictors (300).

Hyperparameter tuning

We tuned `ntree` and `nodesize` on a 5,000-sample subset (computational bottleneck). The tested ranges and selected values are in Table 6.

Name	Range	Selected value
Terminal nodes	[20, 50, 100]	50
Trees	[50, 100, 200]	100

Table 6: Bagging hyperparameters.

Results

Given the memory and time constraints, the final analysis was conducted by training a robust bagging model on a representative 25,000-sample subset of the full dataset. This final bagging model achieved an RMSE of 35.22 on the hold-out validation set.

3.3.3 Random forest

Specification

Random forest was implemented with `ranger` to support parallel training on sparse features.

Hyperparameter tuning

To mitigate computational complexity, hyperparameter tuning was performed on a subset of 30,000 samples. A grid search strategy was employed to optimize three key hyperparameters: Number of Trees (ntree), the Minimum Node Size (nodesize) and the Number of Predictors sampled (mtry). The grid search identified the optimal hyperparameters which are listed in Table 7.

Name	Range	Selected value
Terminal nodes	[5, 10, 20]	5
Trees	[100, 300, 500]	500
Predictors	[10, 17, 25]	25

Table 7: Random forest hyperparameters.

Results

Utilizing the optimal hyperparameters identified during the tuning phase, the final model was retrained on the full training set. This configuration yielded an RMSE of 35.17 on the validation set.

3.3.4 BART

Specification

Bayesian Additive Regression Trees (BART) was implemented using the `dbarts` package, relying on an ensemble of decision trees.

Hyperparameter tuning

Tuning focused on the core structural parameters (ntree and k) using a subset of 30,000 samples. For tuning efficiency, the MCMC parameters (nskip, ndpost) were initially fixed, then increased for the final, robust fit. The configurations tested are detailed below in Table 8.

Name	**Range**	**Selected value**
Number of trees	[50, 200, 500]	50
k (Regularization prior)	[2.0, 3.0, 5.0]	2.0
Burn-in iterations (nskip)	[Fixed]	500
Posterior samples (ndpost)	[Fixed]	2000

Table 8: BART hyperparameters.

Results

The final model, trained on the full dataset with the optimal settings retrieved from tuning, achieved an estimated RMSE of 34.45 on the validation set.

3.3.5 Gradient boosting

Specification

Gradient boosting builds an ensemble of decision trees sequentially, where each tree models the residuals of the previous iteration by following the gradient of the loss function. This iterative correction allows the model to focus on difficult observations and typically yields strong predictive performance compared with independently grown trees.

Hyperparameter tuning

Tuning focused on the main structural parameters, interaction depth and shrinkage, using a subset of 30,000 samples and the 300 lasso-selected predictors. The number of trees was set to a high maximum (3000) and 5-fold cross-validation was used to determine the optimal

stopping iteration, preventing overfitting. Secondary parameters (bagging fraction, minimum node size) were held constant. The tested configurations and selected values are shown in Table 9.

Name	Range	Selected value
Trees	[0-3000]	852
Interaction depth	[3, 5, 7]	7
Shrinkage	[0.01, 0.05, 0.1]	0.1

Table 9: Gradient boosting hyperparameters.

Results

Using this configuration, the model was retrained on the complete training dataset. The final gradient boosting model achieved an RMSE of 34.11 on the hold-out validation set.

3.3.6 XGBoost

Specification

XGBoost is a gradient boosting method optimized for sparse, high-dimensional data. Its sequential tree construction and use of second-order gradients make it well suited for our TF–IDF feature space, offering faster convergence and better regularization than random forest.

Hyperparameter tuning

Hyperparameters were tuned through a randomized search over the ranges shown in Table 10, using 5-fold cross-validation with early stopping (30 rounds). XGBoost also performed substantially better without SVD, as the full sparse TF–IDF matrix preserved useful high-dimensional signal. To further improve accuracy on expensive items, we applied price-based instance weighting: observations above the 80th price percentile received double weight during training.

Name	Range	Selected value
Trees	[MAX 5000, early stopping]	5000
Shrinkage	[0.01, 0.05, 0.1, 0.3]	0.05
Interaction depth	[5, 7, 9, 11, 13]	5
Min child weight	[1, 5, 10]	1
Gamma	[0,1, 3]	1
Subsample	[0.6, 0.8, 1.0]	0.6

Table 10: XGBoost hyperparameters.

Results

Using the selected hyperparameters, a final model was trained on the full training set. This configuration achieved an RMSE of 28.22 on the hold-out validation set, substantially better than the SVD-based variant (30.52), confirming that XGBoost benefits from the full sparse feature representation.

3.3.7 LightGBM

Specification

LightGBM is a gradient boosting framework that grows trees leaf-wise rather than level-wise, selecting the split that maximizes loss reduction. This best-first strategy yields faster training and higher efficiency, making it well suited for large-scale, high-dimensional data such as our TF–IDF feature space.

Hyperparameter tuning

To optimize the LightGBM model, we employed a randomized grid search strategy. A comprehensive search space of 1,620 combinations was defined. From this space, we randomly sampled and evaluated 100 configurations using 5-fold cross-validation on the training set. The optimal hyperparameters resulting from this search are presented in Table 11.

Name	**Range**	**Selected value**
Trees (nrounds)	[MAX 5000, early stopping]	951
Number of leaves	[31, 63 , 127]	31
Shrinkage (learning rate)	[0.01, 0.05, 0.1, 0.3]	0.1
Interaction depth	[5, 7, 9, 11, 13]	13
Min child weight (Hessian)	[1, 5, 10]	10
Gamma (min gain)	[0, 0.1]	0
Feature fraction	[0.6, 0.8, 1.0]	1.0
Bagging fraction	[0.6, 0.8, 1.0]	1.0

Table 11: lightGBM hyperparameters.

Results

The final LightGBM model was retrained on the full training set using the optimal hyperparameter configuration. This yielded a validation RMSE of 28.06, representing a good fit to the data.

3.4 Support vector regression

3.4.1 Linear kernel

Specification

Given the high-dimensional and sparse nature of the TF-IDF features, we employed SVR with a linear kernel due to its robustness against overfitting. The model was implemented using the `LiblineaR` package to allow for efficient training on the full dataset without the need for downsampling.

Hyperparameter tuning

The tuning process of the linear kernel focused on the Cost parameter (C), which regularizes the model by balancing the trade-off between a flat regression line and the training error. We conducted a grid search using 5-fold cross-validation on the training data. The specific values tested and the corresponding selected value are detailed in Table 12.

Name	**Range**	**Selected value**
Kernel	["linear"]	"linear"
Cost C	[0.01, 0.1, 1, 10, 100]	1

Table 12: SVR with linear kernel hyperparameters.

Results

Using the optimal Cost parameter identified during tuning, the linear SVR achieved a validation RMSE of 33.10.

3.4.2 Non-linear kernel

Specification

To capture non-linear patterns, we employed SVR with non-linear kernels. Due to quadratic computational scaling, we utilized a subsetting strategy: tuning on 5,000 observations and training on 50,000.

Hyperparameter tuning

We conducted a 3-fold cross-validation grid search on the tuning subset to optimize the Cost (C) for both kernels, alongside Gamma (γ) for radial and Degree (d) for polynomial. The tested ranges and optimal values are summarized in Table 13.

Results

Using the optimal parameters, the final model was trained on the 50,000-observation subset. This configuration resulted in a validation RMSE of 31.78.

Name	Range	Selected value
Kernel	[“polynomial”, “radial”]	radial
Cost C	[0.1, 1, 10, 50, 100]	10
Degree d	[2, 3]	2
Gamma γ	[0.0001, 0.001, 0.01, 0.1, 1]	0.01

Table 13: SVR with non-linear kernel hyperparameters.

3.5 Neural networks

3.5.1 Multi-layer perceptron

Specification

The MLP provides a nonlinear baseline for price prediction and uses a dense 150-dimensional SVD embedding of the word-level TF–IDF features from the concatenated product description and name, offering a richer semantic representation than the 100-dimensional SVD features used in the other models. The network uses three dense layers with GELU activation, L2 regularisation and dropout, and is trained with Adam and Huber loss.

Hyperparameter tuning

Hyperparameters were selected by grid search with 3-fold cross-validation and early stopping. Using the best configuration (Table 14), we retrained on the full training set with `ReduceLROnPlateau` and early stopping, repeating training five times with different random seeds and averaging validation RMSE.

Results

Figure 2 shows that training and validation loss decrease smoothly and in parallel, with the validation curve levelling off slightly earlier. This is evidence of effective regularisation and the absence of overfitting. Learning-rate reductions correspond to plateaus in validation loss and produce a stable, gradually slowing convergence trajectory, indicating reliable optimisation and good generalisation.

Name	Range	Selected value
Architecture		
Hidden layers	[1, 2, 3]	3
Hidden units (Layer 1)	[128, 196,212, 256]	212
Hidden units (Layer 2)	[64, 96]	96
Hidden units (Layer 3)	[16, 32, 64]	16
Activation function	[relu, tanh, gelu]	gelu
Regularisation		
Dropout rate (Layer 1)	[0.1,0.18, 0.2,0.22]	0.18
Dropout rate (Layer 2)	[0.0, 0.05,0.15, 0.20]	0.05
L2 regularisation	$[1 \cdot 10^{-4}, 3 \cdot 10^{-4}, 3 \cdot 10^{-5}]$	$3{\cdot}10^{-5}$
Training		
Learning rate	$[3 \cdot 10^{-3}, 1 \cdot 10^{-3}]$	$1 \cdot 10^{-3}$
Batch size	[256]	256

Table 14: Search space of the MLP hyperparameters.

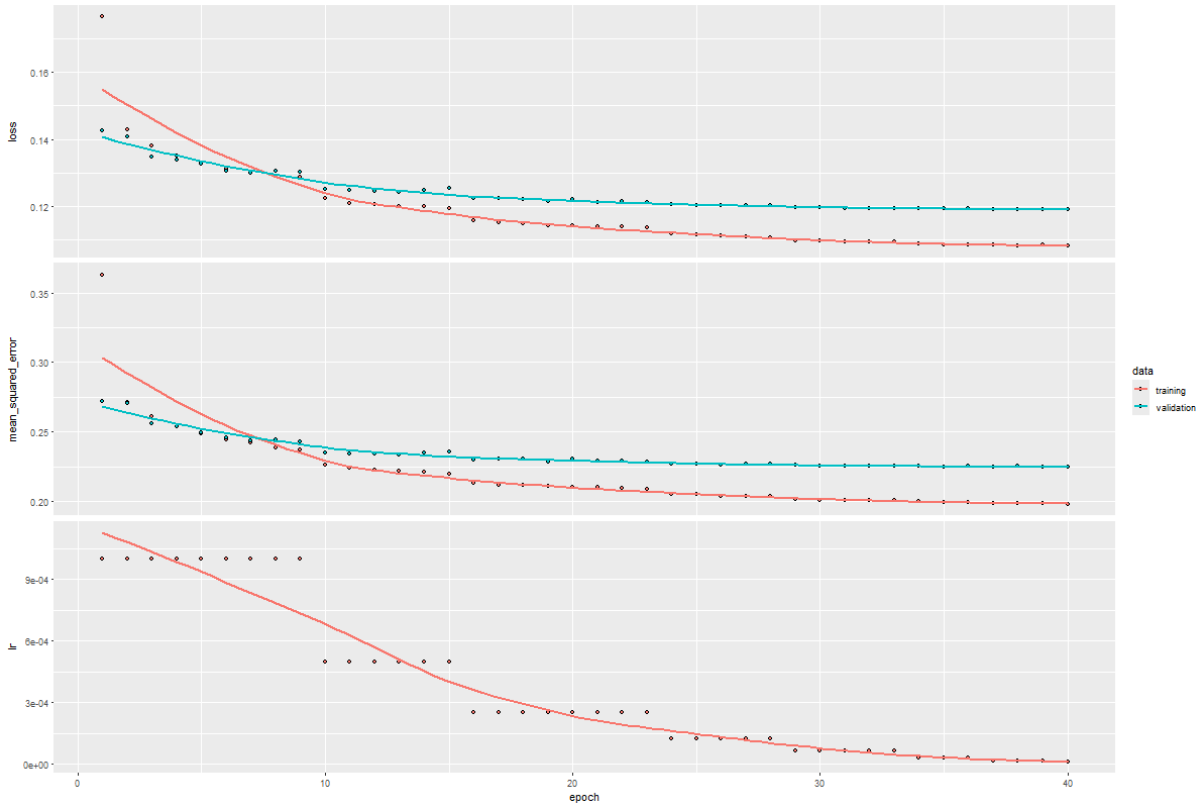


Figure 2: MLP learning convergence.

4 Model Evaluation

Table 15 summarizes validation RMSE across models; MLP and boosting methods perform best.

Model	Val. RMSE
Linear baselines	
Linear regression	32.91
Ridge regression	32.92
Lasso regression	32.94
Generalized additive models	
Smoothing splines	32.87
Tree-based models	
Decision tree	34.85
Bagging	34.66
Random forest	35.17
BART	34.45
Gradient boosting	34.11
XGBoost	28.22
LightGBM	28.06
Support vector regression	
Linear kernel	33.10
Non-linear kernel	31.78
Neural networks	
Multi-layer perceptron	26.12

Table 15: Validation set metrics.

5 Conclusion

The MLP and a weighted XGBoost model achieved the strongest individual performance in this study. Rather than selecting a single model, we combined both using a stacking approach in which a generalized additive model (GAM) was fitted on their validation predictions to learn a nonlinear combination. The smoothness of the GAM terms (i.e., the effective degrees of freedom) was automatically selected via penalized likelihood optimization using the mgcv package. The resulting stacked model outperformed both base learners, achieving a lower validation RMSE (25.52).

These results indicate that MLP and XGBoost capture complementary structure in the data, and that a lightweight stacking layer can successfully integrate their strengths to improve predictive accuracy.

Appendix C visualizes the predictive behaviour of the individual models and the stacked GAM.

The stacked model shows closer alignment with the identity line in scatterplots, reduced systematic residual patterns, and clear complementarity between the MLP and XGBoost predictions.

A Preprocessing and Feature Engineering

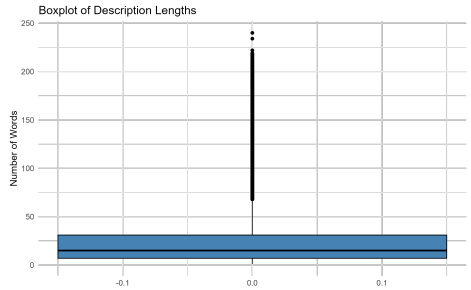


Figure 3: Boxplot of Description Lengths.

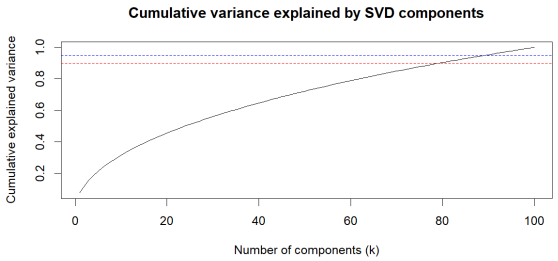


Figure 4: Explained cumulative variance used for SVD

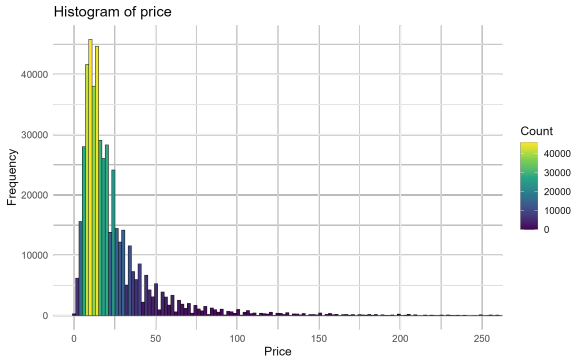


Figure 5: Distribution of price

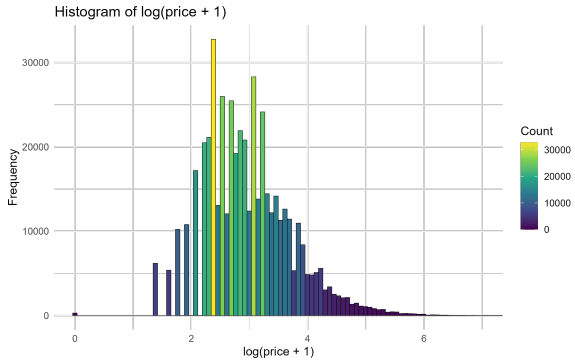


Figure 6: Distribution of the log(price)

B Models

B.0.1 GAM

Variable	AIC (GAM)	ΔAIC
svd_text_2	877,984	6,344
svd_text_6	879,073	4,315
svd_text_24	880,782	4,227
svd_text_5	881,934	3,144
desc_word_count	881,974	2,866
svd_text_3	876,103	2,697
svd_text_19	880,250	2,060
svd_text_12	883,144	1,985
svd_text_8	882,785	1,817
svd_text_68	881,401	1,870

Table 16: Top-10 AIC improvements of spline over linear effects.

Predictor	F-score	p
svd_text_2	246.1	$< 10^{-16}$
svd_text_6	698.7	$< 10^{-16}$
svd_text_5	252.9	$< 10^{-16}$
svd_text_24	643.4	$< 10^{-16}$
desc_word_count	569.6	$< 10^{-16}$

Table 17: Spline significance (nonparametric F).

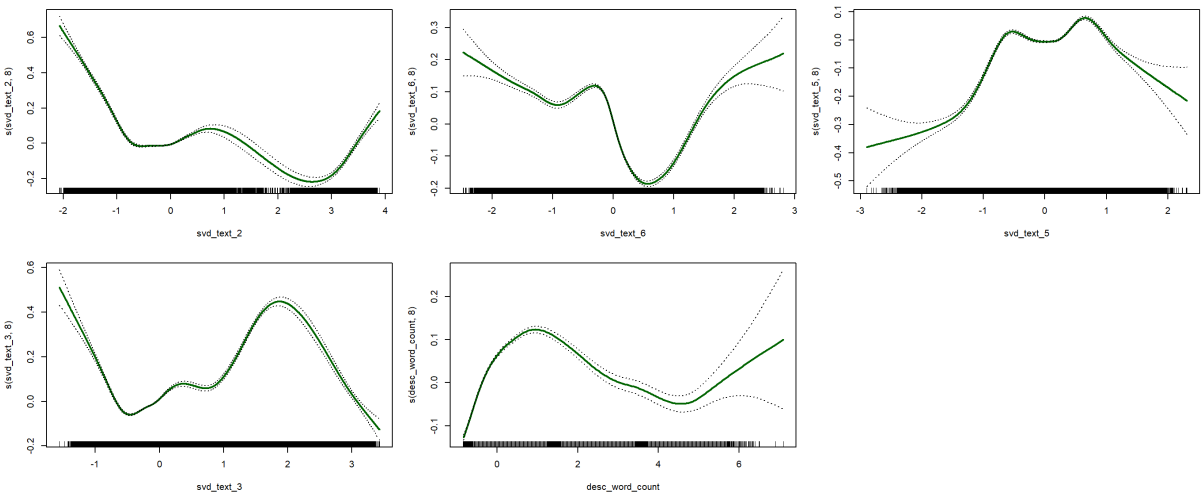


Figure 7: Estimated smoothing splines for the five selected predictors in the GAM.

C Evaluation

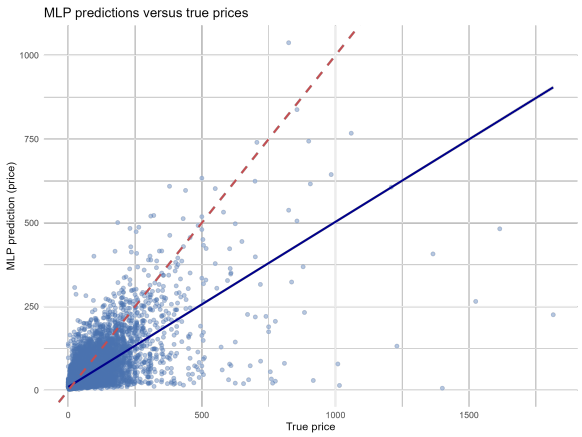


Figure 8: MLP: Prediction vs. Target

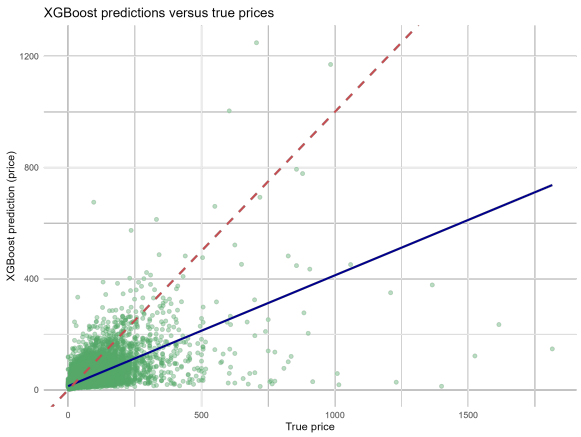


Figure 9: XGBoost: Prediction vs. Target

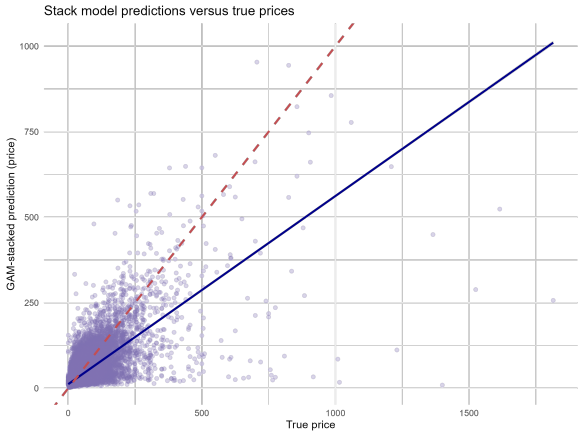


Figure 10: GAM Stack: Predicted vs. True Prices

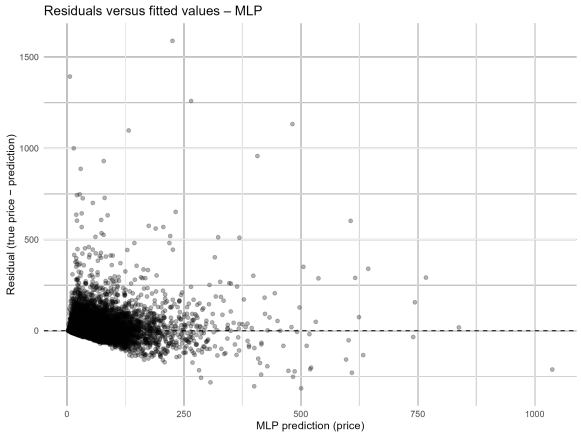


Figure 11: MLP: Residuals vs. Fitted

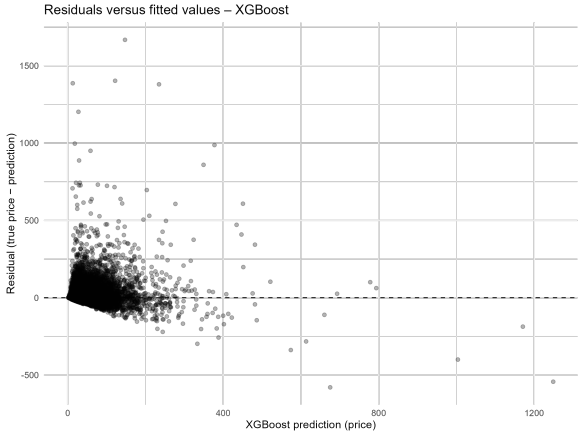


Figure 12: XGBoost: Residuals vs. Fitted

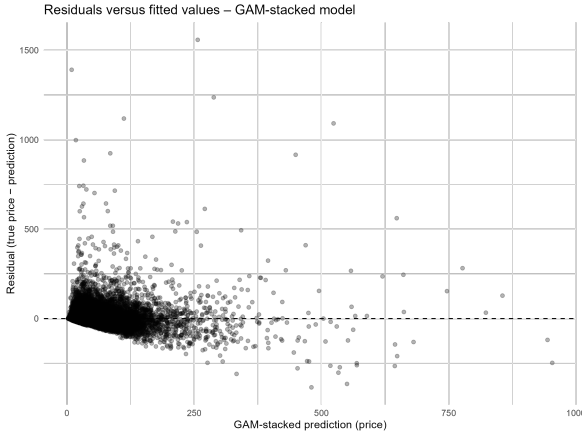


Figure 13: GAM Stack: Residuals vs. Fitted